```
/*    CIRCBUF.C                                                          */
/*    This is an echo program written in C for the ADSP-21061 EZ-KIT Lite.  The  */
/*    echo program takes the talkthru program and adds a circular buffering scheme.    */
/*    The circular buffer is defined by the functions CIRCULAR_BUFFER, BASE,           */
/*    and LENGTH.  The echo is performed by adding the current input to the oldest     */
/*    input. The delay in the echo can be modified by changing BUFF_LENGTH.    */
/*                                                                                     */
/*                                                                                     */
#include <21020.h>          /*  for the idle() command  */
#include <signal.h>         /*  for the interrupt command  */
#include <macros.h>         /*  for the CIRCULAR_BUFFER and segment functions  */

#define BUFF_LENGTH 4000
                                        /*  define echo as 21k DAG1 reg i1  */
CIRCULAR_BUFFER (float,1,echo)          /*  a DM pointer to a circular buffer  */

volatile float in_port segment (hip_reg0);      /*  hip_reg0 and hip-reg2 are  */
volatile float out_port segment (hip_reg2);     /*  used in the architecture file  */

void process_input (int);
void main (void)

{     /*  Make this a variable length array.  If emulator is stopped at main      */
      /*  and _BUFF_LENGTH in dm window is modified, the echo delay                */
      /*  is modified.  Do not make BUFF_LENGTH greater than stack size!           */

float data_buff [BUFF_LENGTH];
interrupt (SIG_IRQ3, process_input);

BASE (echo) = data_buff;                /*  Loads b1 and i1 with buff start adr  */
LENGTH (echo) = BUFF_LENGTH;            /*  Loads L1 with the length of the buffer */

      /*  as the array is filled, the nth location contains the newest value, while  */
      /*  the nth + 1 location contains the oldest value.  */

while (1)
 {                                      /*  the echo sends the sum of the most  */
  float oldest, newest;                 /*  recent value and the oldest value  */
  idle();

      /*  Echo is pointing to the nth location after the interrupt routine.     */
      /*  Place the new value in variable 'newest'.  After the access, update   */
      /*  the pointer by one to point at location n+1.                          */
  CIRC_READ (echo, 1 newest, dm);

      /*  Now echo is pointing to n+1.  Read the location and place value in    */
      /*  variable 'oldest'.  Do not update the pointer, since it is now        */
      /*  pointing to the new location for the interrupt handler.               */
  CIR_READ (echo, 0, oldest, dm);

      /*  add the oldest and most recent and send out on port                   */
  out_port=oldest+newest;
 }
}

void process_input (int int_number)
{
      /* The newest input value is written over the oldest value in the nth  */
      /*  location and the pointer is not updated.                           */
  CIRC_WRITE (echo, 0, in_port, dm);
}
```

TABLE 29-4